



IDS Alert Prioritization and Strengthening

The University of Texas at Dallas

Asad-ullah Khan

Vishal Madipadga

Skye Pekerti

Pratidhi Pratyasha



Project Description

- Intrusion Detection Systems (IDS) require a lot of manual tweaking to be effective but can still report a lot of false positives
- Determine effective ways to use machine learning to improve the performance of an IDS
- CICFlowMeter - CSE-CIC-IDS 2018



Data Setup

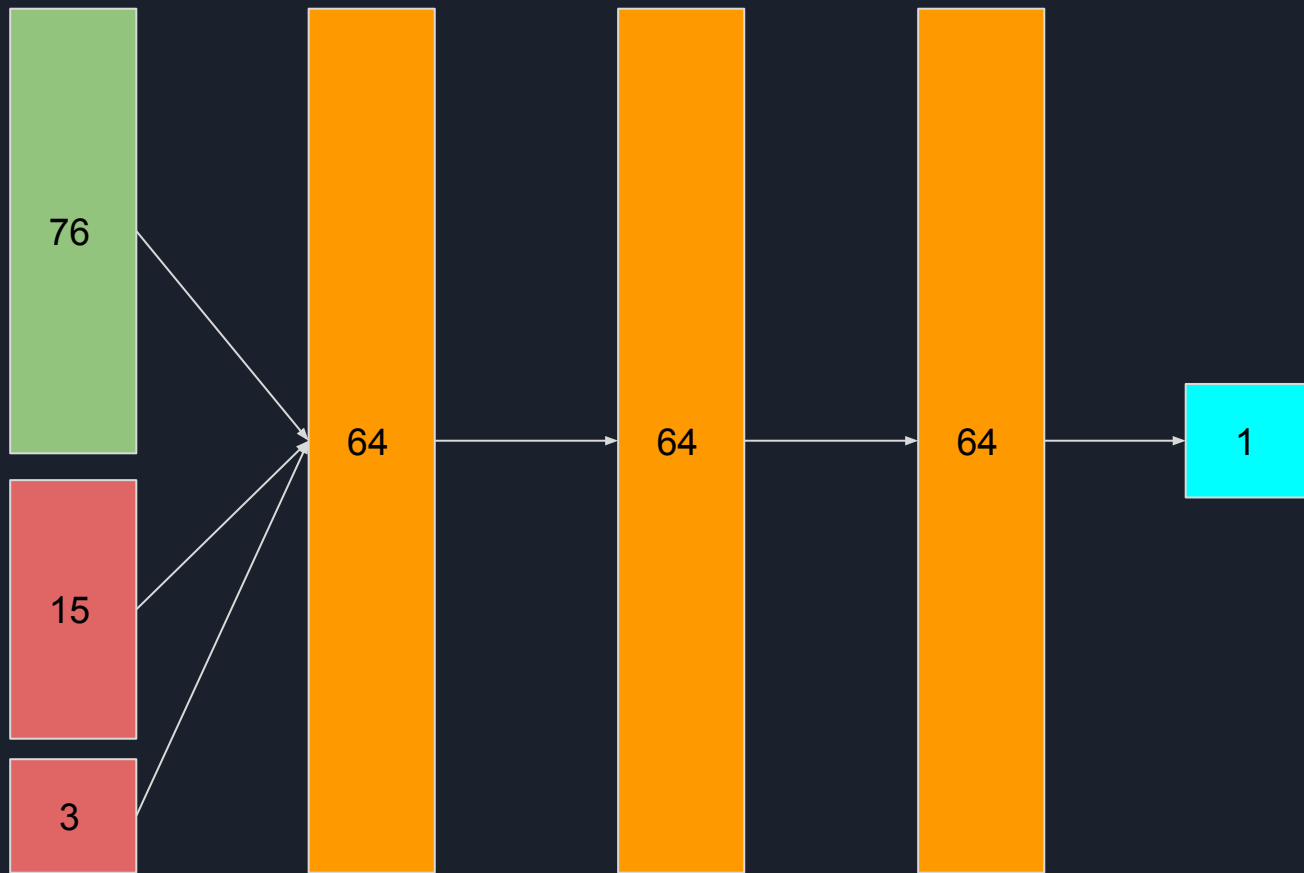
- The data we used was provided in multiple CSV files, totaling around 6.41 GB.
- We merged them into one file as an hdf5, making it easier to work with in TensorFlow.
- The main advantage of using this format is speed. It can:
 - Load the entire dataset in less than a seconds
 - The resulting data is a NumPy array, which can then be fed into the data generator
 - The entire dataset can be scrambled, sorted, normalized, etc. at will
- While creating the data, there were several issues:
 - Duplicate CSV headers in the file
 - NaN and Infinity scattered throughout some of the columns
 - Some columns did not have any values other than 0.0, and were discarded
 - Calculating the minimum and maximum for each column (for normalization) was time consuming
- However, once these issues were overcome, they never had to be visited again



Basic model

- This model used a simple structure of:
 - Input
 - 3x hidden 64 unit layers
 - Output
- We used an embedding layer which reduced the 65,000+ categorical dimension of the input port down to 15, and then concatenated this along with the rest of the continuous data

Basic DNN

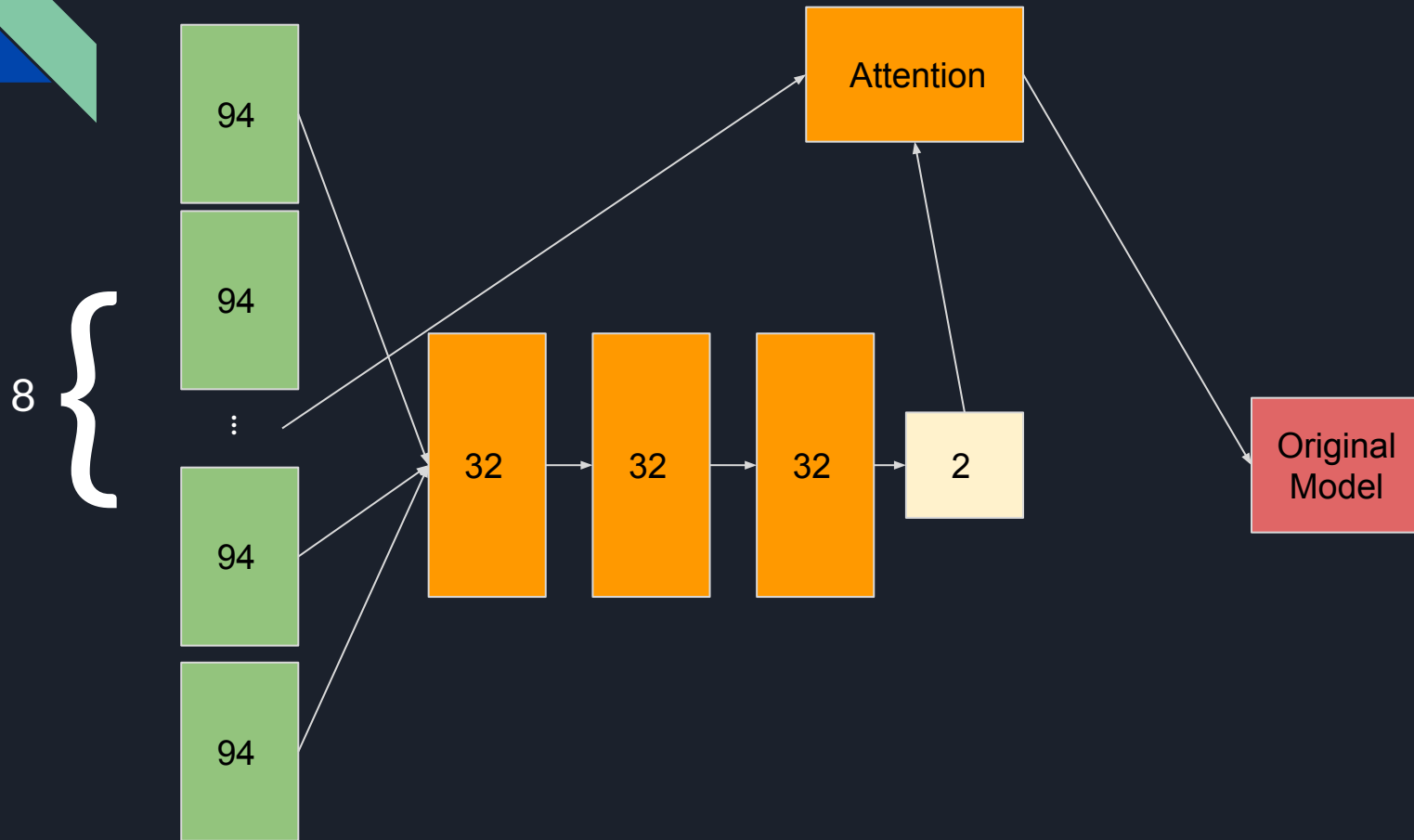




Attention Model

- By analyzing not just the current interaction, but also some previous interactions simultaneously, we hypothesize that we can improve the accuracy of the model
- Using this technique, we improved the true positive rate of the previous model from 95% to 97%
- The model architecture is a bit more complex, however

Attention architecture





Feature Reduction

- The dataset we used had 80 columns, meaning we had 80 features
- We wanted to try to reduce the number of features we used in order to improve the performance of our Neural Network
- Benefits of feature reduction:
 - Ability to throw away features that are useless (very low information gain)
 - Training our Neural Network would be less computationally expensive since we would have less inputs to the Neural Network
 - Possible improved performance of classifiers that suffer from the curse of dimensionality such as K Nearest Neighbors

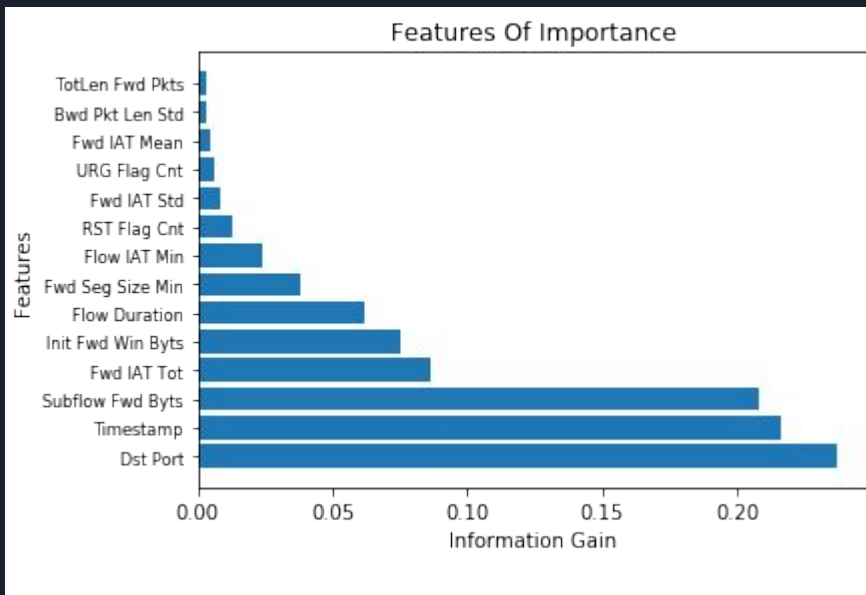


Feature Reduction using Decision Trees

- We chose to employ a decision tree classifier in order to figure out which features were more important
- Decision trees attempt to split data into homogeneous groups and they do this by choosing to a certain feature value to split the data on
- Decision trees choose which features to split the data on based on the information gain achieved by a feature

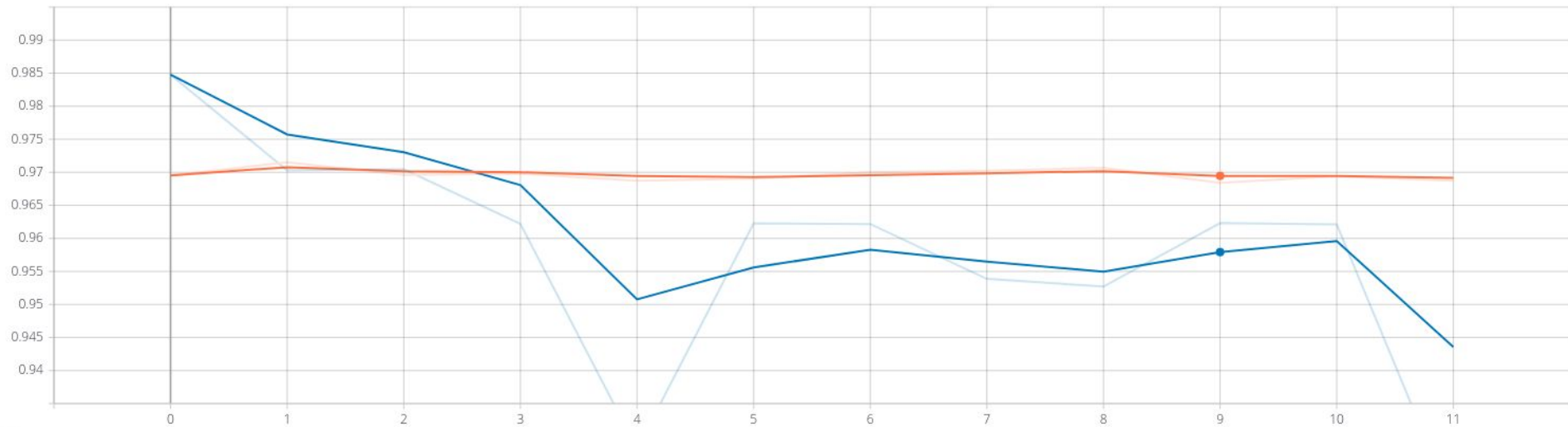
Features of Importance

- We removed features with an information gain value less than 10^{-3}
- This left us with 14 features



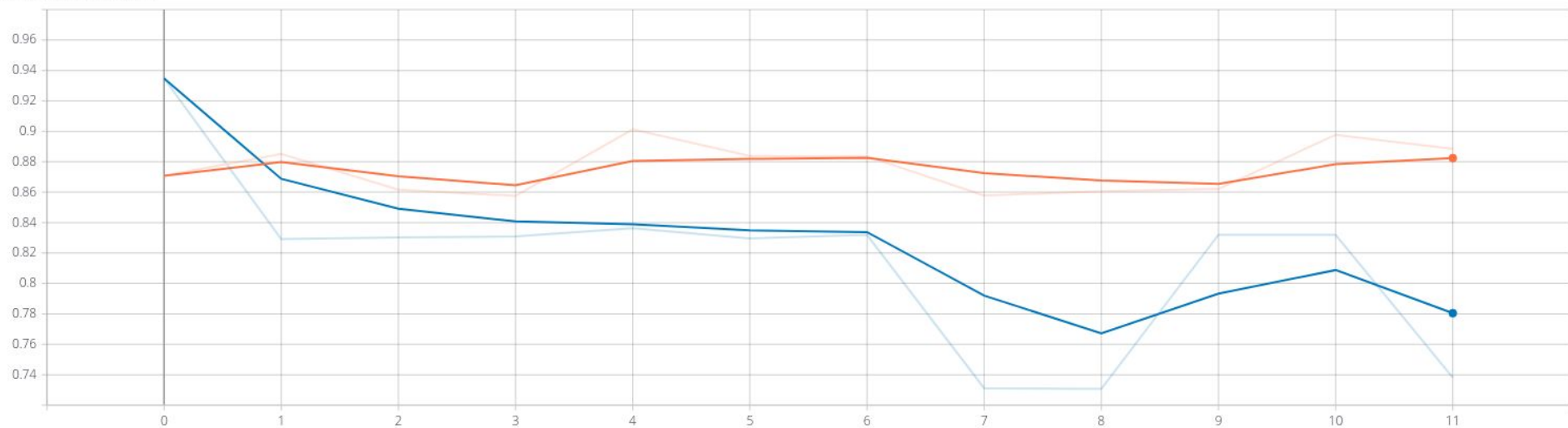
Results - Basic Model

epoch_binary_accuracy



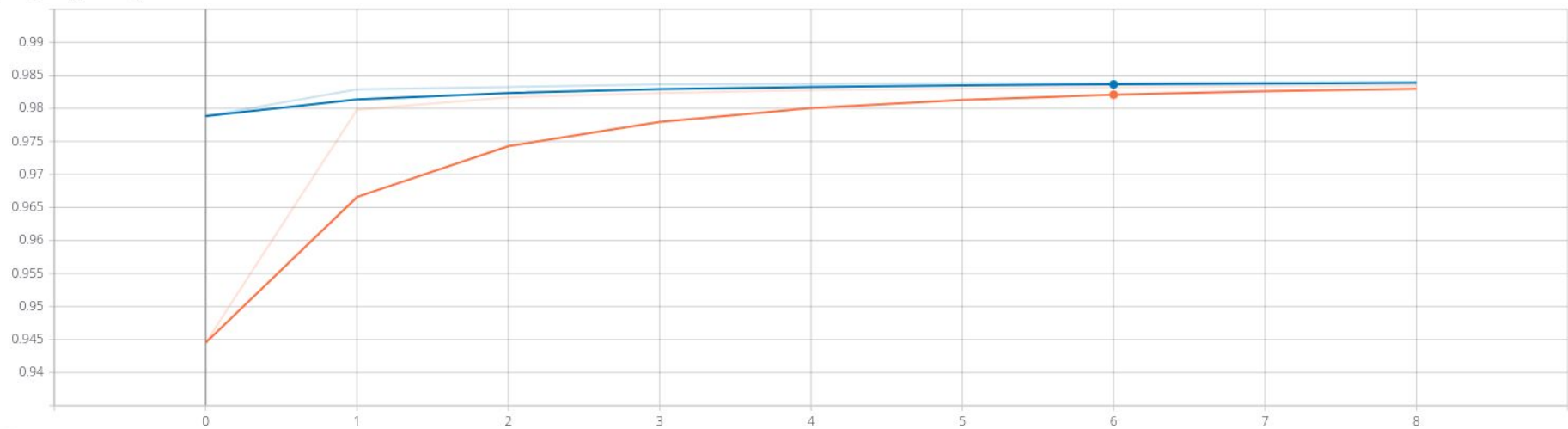


epoch_true_positive_rate



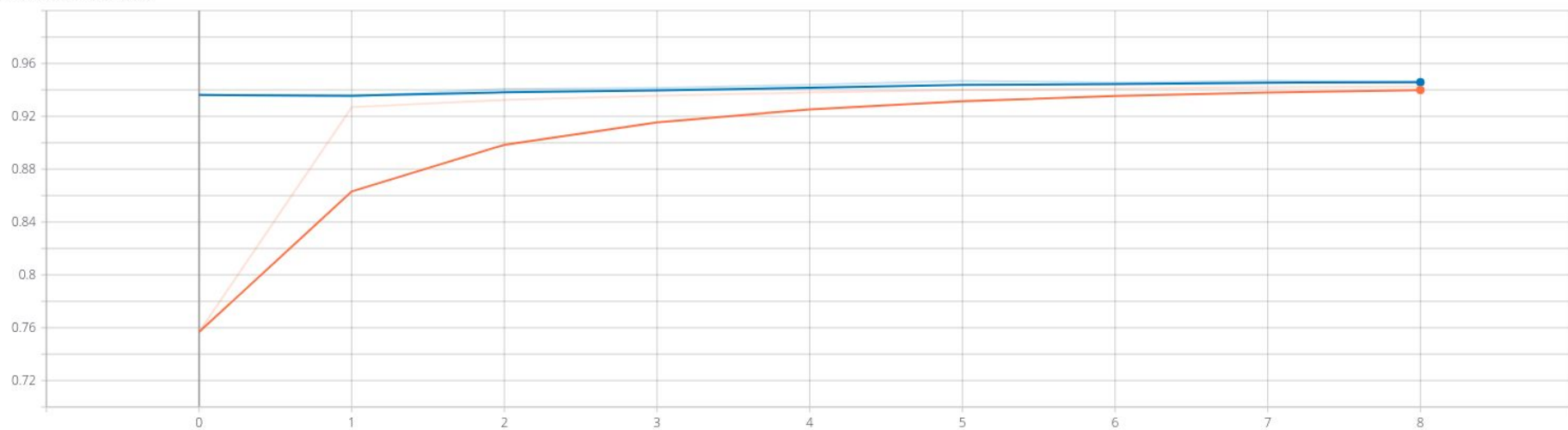
Results - Attention Model

epoch_binary_accuracy





epoch_true_positive_rate



Results - Decision Tree





Issues and Challenges

- There were a lot of problems regarding the datasets we used. A lot of time was spent attempting to clean up and fix datasets so they were workable.
- Due to the size of the dataset, it took a long time to build some classifiers (usually a few days)
- We are using a Python library called scikit-learn which allows us to implement different machine learning classifiers
 - One problem we had was getting the data to be the right format so that it could be used with scikit-learn
 - scikit-learn does not allow for the use non-numeric feature so we have had to filter out all non-numeric features such as Source IP address and Destination IP address



Future Work

- Build classifiers using the reduced feature set and see if there is an improvement over the same classifiers using the original feature set
- Test different thresholds when determining which features to leave out
 - Then build a classifier based on these features and compare their performance
- Run the models against further sets of data to test accuracy.
 - Potentially see if the models can detect malicious packets in a simulated or real environment as opposed to datasets.
- Unsupervised learning via autoencoders, kNN, siamese learning